Annexe A21 — Leçon : le chantier B, la vectorisation

LibraryV3 · Phase G, étape 4b · préparation · ce que fait le pipeline par sommet et par face, et pourquoi la disposition des données décide de ce que l'AVX2 peut faire

---

En une ligne. La vectorisation ne consiste pas à « mettre des intrinsèques » dans le code existant : elle consiste à disposer les données pour que **la même instruction s'applique à 8 éléments contigus**. Ce chantier commence donc par la mémoire (positions en SoA), garde une image **identique au bit près** tant qu'on ne décide pas explicitement le contraire, et se juge sur une scène où les faces dominent (campagne V), jamais sur la Référence L.

Cette leçon prépare le chantier. Les chiffres viendront de la référence V0, en cours de mesure ; chaque étape proposée ici sera jugée par une paire A/B dans un même lot.

1. Ce que fait aujourd'hui le pipeline, pour un mesh `Inside`

Pour chaque paire (objet, vue) qui n'est ni rejetée ni dessinée en point :

Étage sommets (`ProjectPositions`, une fois par sommet, chantier 2b) :

```
for (size_t i = 0; i < count; ++i)
{
    const Vec4f c = MulRow(m, src[i]);        // 12 multiplications + 12 additions
    dst[i] = ProjectClip(c, v);               // 1 division, 3 multiplications, ToRaster
}
```

Étage faces (`RenderView`, une fois par face) :

```
for (size_t f = 0; f < nFaces; ++f)
{
    const uint32_t* idx = &mesh->indices[f * vpf];           // 3 indices
    EmitRasterTriangle(renderer, view, rv[idx[0]], rv[idx[1]], rv[idx[2]], col);
}
// EmitRasterTriangle : aire signee (EdgeFunction) -> face de dos ? -> boite serree vide ? ->
DrawTriangle
```

Les données :

Donnée	Type	Disposition
positions	`std::vector<Vec3f>`	AoS, 12 octets par sommet
indices	`std::vector<uint32_t>`	3 par face
sommets projetés	`RasterVertex[]` (x, y, z, invW)	AoS, 16 octets alignés

La campagne R a chiffré l'ensemble à **14,4 ns par face soumise** (avec sa part de sommets), soit environ 60 cycles à 4 GHz, et 92 ns de coût fixe par objet dessiné. La campagne V dira quelle part revient aux sommets et quelle part aux faces.

2. Ce qu'est un registre SIMD

Jeu	Registre	Largeur	Floats par registre	Machines
SSE2	`xmm`	128 bits	4	partout (x64)
AVX2 + FMA	`ymm`	256 bits	**8**	i7-1265U, i9
AVX-512	`zmm`	512 bits	16	i9 seulement

Une instruction SIMD applique **la même opération** aux 8 voies d'un registre : `_mm256_add_ps(a, b)` fait 8 additions indépendantes. Le gain vient de là, et seulement de là : il faut **8 éléments indépendants** sur lesquels faire la même chose.

Dans notre pipeline, ces éléments existent : 8 sommets différents subissent exactement la même transformation, et 8 faces différentes subissent exactement le même test.

3. Horizontal ou vertical : le contre-exemple classique

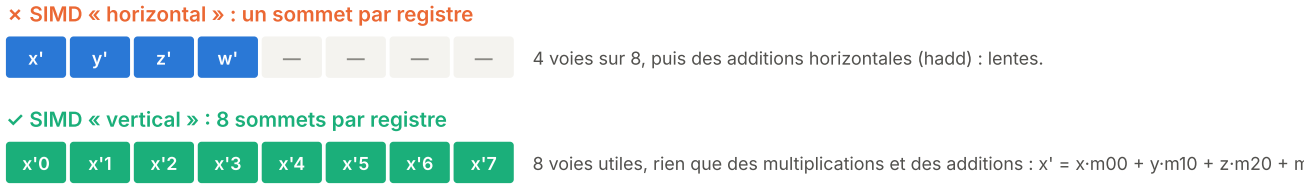


Figure 2 — Le contre-exemple classique : vectoriser l'INTÉRIEUR d'un sommet (MulRow sur 4 voies). On vectorise ENTRE les sommets.

> × Vectoriser l'**intérieur** d'un sommet : charger (x, y, z, 1) dans un registre de 4 voies et calculer `MulRow` avec 4 produits « ligne × colonne ». On n'utilise que 4 voies sur 8, et chaque composante du résultat exige une somme **horizontale** (`hadd`), lente. C'est la première idée qui vient, et elle est presque toujours plus lente que le scalaire bien compilé.

La bonne forme est **verticale** : un registre contient la même composante de 8 sommets. La transformation devient 12 opérations, chacune sur 8 sommets à la fois :

```
x'[0..7] = x[0..7]·m00 + y[0..7]·m10 + z[0..7]·m20 + m30      (et de même pour y', z', w')
```

4. La disposition mémoire décide

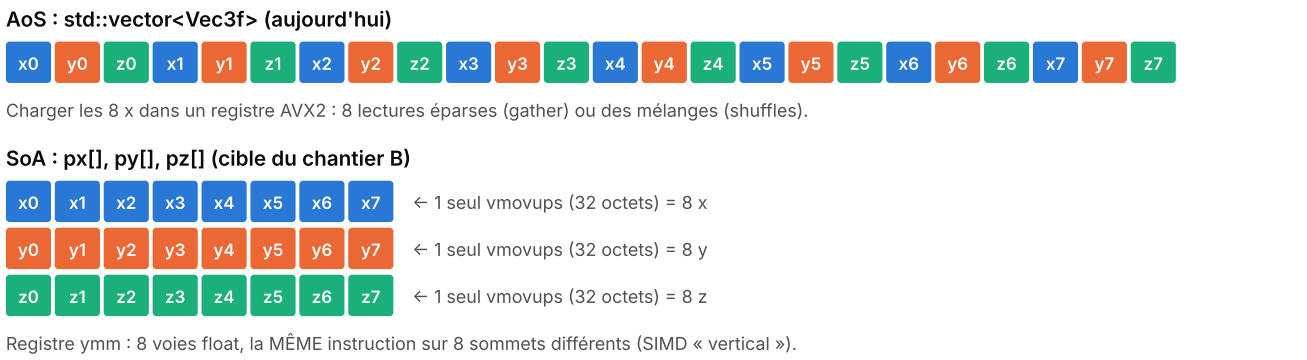


Figure 1 — La disposition mémoire décide de ce que l'AVX2 peut faire. En AoS, les 8 x utiles sont espacés de 12 octets ; en SoA, ils sont contigus.

Pour remplir un registre avec `x[0..7]`, il faut que les 8 x soient **contigus** en mémoire. En AoS (`std::vector<Vec3f>`), ils sont espacés de 12 octets : il faut soit 8 lectures éparses (`vgatherdps`, coûteux sur Intel), soit charger 3 registres et les mélanger (shuffles), ce qui coûte presque autant que le calcul. En **SoA** (`px[]`, `py[]`, `pz[]`), un seul `vmovups` charge 8 x.

C'est le cœur de l'approche DOD du projet : **on choisit la disposition pour l'algorithme qui la lit, pas pour la commodité de celui qui l'écrit.** La conversion AoS → SoA se fait **une fois, au chargement** du mesh (`OBJLoader` ou `RegisterMesh`), jamais par frame.

Deux détails de la disposition :

- **Alignement sur 32 octets** (64 pour l'AVX-512) : un chargement aligné ne traverse jamais deux lignes de cache.
- **Bourrage à un multiple de 8** : on alloue `ceil(n / 8) × 8` sommets, en répétant le dernier. La boucle n'a plus de **queue scalaire**, donc plus de deuxième chemin de code qui pourrait diverger.

> × Convertir AoS → SoA à chaque frame, dans `RenderView`. On paierait à chaque frame une copie de toutes les positions pour gagner sur leur transformation : le gain disparaît dans la copie.

5. L'étage sommets en AVX2 : une illustration

Ce n'est pas encore le code à intégrer : c'est la forme qu'il prendra.

```
// Constantes : une fois par (objet, vue), hors de la boucle (16 coefficients en tout)
const __m256 m00 = _mm256_set1_ps(m[0][0]), m10 = _mm256_set1_ps(m[1][0]);
const __m256 m20 = _mm256_set1_ps(m[2][0]), m30 = _mm256_set1_ps(m[3][0]);  // ... colonnes 1 a
3
const __m256 one = _mm256_set1_ps(1.0f),  half = _mm256_set1_ps(0.5f);
const __m256 vpX = _mm256_set1_ps(float(vp.x)), vpW = _mm256_set1_ps(float(vp.width));

for (size_t i = 0; i < nPadded; i += 8)           // nPadded : multiple de 8, aucune queue
{
    const __m256 x = _mm256_load_ps(px + i);       // 8 x contigus, alignes sur 32 octets
    const __m256 y = _mm256_load_ps(py + i);
    const __m256 z = _mm256_load_ps(pz + i);

    // MEME ordre d'operations que MulRow : ((x*m00 + y*m10) + z*m20) + m30
    __m256 cx = _mm256_add_ps(_mm256_mul_ps(x, m00), _mm256_mul_ps(y, m10));
    cx = _mm256_add_ps(_mm256_add_ps(cx, _mm256_mul_ps(z, m20)), m30);
    // ... cy, cz, cw de meme (colonnes 1, 2, 3)

    const __m256 inv = _mm256_div_ps(one, cw);     // division IEEE exacte, comme 1.0f / w

    // MEME ordre que Viewport::ToRaster : vp.x + (xn*0.5 + 0.5) * width
    const __m256 xn = _mm256_mul_ps(cx, inv);
    const __m256 xr = _mm256_add_ps(vpX, _mm256_mul_ps(_mm256_add_ps(_mm256_mul_ps(xn, half),
half), vpW));
    _mm256_store_ps(rx + i, xr);                   // ... ry (0.5 - yn*0.5), rz = cz*inv, rinvW
= inv
}
```

5.1 Le bit près : pourquoi pas de FMA tout de suite

Une **FMA** (`_mm256_fmadd_ps`) calcule `a·b + c` avec **un seul** arrondi, au lieu de deux. Elle est plus rapide et plus précise, mais elle ne donne pas **le même** résultat au dernier bit. Or le chantier 2b a posé une règle : `ProjectClip` est la seule formule de projection, utilisée par le chemin `Inside` (par sommet) et par le chemin `Intersect` (par coin, après clipping), « donc une image identique au bit près entre les deux

chemins ». Un objet qui passe de `Intersect` à `Inside` en bougeant ne doit pas voir ses pixels de bord changer.

D'où la discipline du chantier :

1. **D'abord bit-exact** : le code vectoriel reproduit l'ordre des opérations scalaires (mul puis add, division exacte). La TNR compare les images **pixel par pixel** : 0 pixel différent est le seul résultat accepté.
2. **Ensuite, et séparément**, si on veut la FMA : on l'applique **aux deux chemins** en même temps (scalaire et vectoriel), on mesure le gain, et on accepte explicitement une nouvelle image de référence.

> ✗ Laisser le compilateur « contracter » `a*b + c` en FMA dans un chemin et pas dans l'autre (`/fp:contract`, ou du code AVX2 écrit avec `fmadd` face à un scalaire qui ne l'a pas). Les deux chemins divergent au dernier bit, la règle top-left tranche différemment sur un bord exact, et un pixel clignote quand un objet change de chemin.

> ✗ Remplacer la division par `_mm256_rcp_ps` (inverse approché, 12 bits) « parce que c'est plus rapide ». C'est un changement d'image, pas une optimisation neutre : il se décide et se mesure comme la FMA, jamais en passant.

6. L'étage faces : rejeter 8 faces à la fois

Après la projection, chaque face passe trois tests : face de dos, boîte de pixels vide, puis dessin. Dans les scènes à petits objets, **la grande majorité des faces s'arrête aux deux premiers tests**. Ce sont donc eux qu'il faut vectoriser, et laisser le dessin (rare) en scalaire.

Schéma pour un lot de 8 faces :

1. lire 8 × 3 indices (contigus : 24 `uint32_t`) ;
2. récupérer x et y des 3 coins pour 8 faces : avec des sommets projetés en SoA (`rx[]`, `ry[]`), ce sont 6 lectures indexées (`_mm256_i32gather_ps`) ;
3. calculer 8 aires signées et 8 boîtes serrées ;
4. former un **masque** des faces survivantes (`_mm256_movemask_ps`) ;
5. ne traiter **en scalaire** que les faces dont le bit est à 1.

```
int mask = _mm256_movemask_ps(survivantes);       // 8 bits
while (mask) {
    const int k = std::countr_zero(unsigned(mask)); // indice de la face survivante
    EmitSurvivor(base + k);                          // chemin scalaire existant
    mask &= mask - 1;                                // efface ce bit
}
```

La lecture indexée (`gather`) est l'opération chère du schéma : sur Intel, elle coûte environ un élément par cycle. Ce sera la question de mesure de cette étape : le gain des tests vectoriels couvre-t-il le coût des lectures indexées ? Le résultat n'est pas connu d'avance.

7. Ton idée : le test des faces de dos en espace objet

La technique décrite par scratchapixel : transformer **une fois** la position de la caméra dans l'espace de l'objet, puis tester chaque face **avant** toute projection.

```
oeilLocal = M⁻¹ · oeilMonde                                  // 1 fois par (objet, vue)
face f visible  ⇔  n_f · (oeilLocal − a_f)  du bon signe  // a_f : un sommet de f, n_f : normale de f
```

Pourquoi c'est exact. Le signe d'un point par rapport à un plan est conservé par toute transformation affine qui conserve l'orientation : on peut donc faire le test dans l'espace de l'objet plutôt qu'en espace monde. Deux exceptions :

- une matrice **miroir** (déterminant de la partie 3 × 3 négatif) inverse le signe : on le lit une fois par objet ;
- en **orthographique**, l'œil est à l'infini : on teste avec la **direction** de vue ramenée en local (`n_f · dirLocal`).

Ce que cela demande. Les plans des faces (`n`, `d`) précalculés au chargement, en SoA : 16 octets par face. Le test de 8 faces coûte alors 3 FMA et une comparaison.

Ce que cela rapporte, et ce que cela coûte. Environ la moitié des faces est écartée **avant** l'étage faces. Mais :

Question	Réponse
Évite-t-il la transformation des sommets ?	Non, si l'étage sommets reste un traitement en lot de tous les sommets (§ 5). Il faudrait une transformation « à la demande », qui casse le lot.
Est-il bit-exact avec le test actuel (aire signée en raster) ?	**Non.** Pour une face presque de profil, les deux tests peuvent trancher différemment. Ces faces n'allument presque rien, mais des pixels de silhouette peuvent changer. Il faudra compter les pixels différents.
Concurrence-t-il le § 6 ?	Oui : les deux écartent les mêmes faces de dos. On mesurera les deux, et peut-être leur combinaison.

> **À retenir.** Le test en espace objet est un **vrai** levier pour les faces de dos ; il n'apporte rien pour un objet **hors champ**, déjà rejeté en 19 ns par sa boîte (A19). Et plus l'étage sommets devient bon marché grâce au SIMD, moins il rapporte d'éviter des sommets. C'est pourquoi il se mesure contre la transformation en lot, pas seul.

8. Construire le binaire : AVX2 partout, AVX-512 à l'exécution

- `/arch:AVX2` **pour tout le moteur**, posé dans un seul fichier (`LV3.Common.props`, règle M6). Tout le code est alors encodé en VEX, le compilateur place seul les `vzeroupper`, et les contraintes manuelles rencontrées dans le chantier Clear (A15 § 6.3) disparaissent. Supporter une machine sans AVX2 n'est pas un objectif.
- **AVX-512 en variante**, dans une unité de compilation séparée (`/arch:AVX512`), choisie **à l'exécution** par une détection CPUID au démarrage : l'i7-1265U n'a pas d'AVX-512, et une instruction AVX-512 y lèverait une instruction illégale. Le choix se fait par un pointeur de fonction, une fois.

> ✗ Un `#ifdef __AVX512F__` global. Le binaire compilé pour l'i9 plante sur l'i7, et le binaire compilé pour l'i7 n'utilise jamais l'AVX-512 de l'i9.

9. Le plan du chantier

Pas	Contenu	Mesure	Critère
B0	Référence V0 (en cours)	V1, V2	point zéro

Pas	Contenu	Mesure	Critère
B1	`/arch:AVX2` pour tout le moteur, **aucun code changé**	V1, V2, Référence L	ce que le compilateur fait seul ; image identique
B2	positions en SoA, alignées et bourrées, au chargement ; code **scalaire** lisant le SoA	V1	neutre attendu (on change la mémoire, pas le calcul)
B3	étage sommets en AVX2, bit-exact	V1	0 pixel différent ; gain sur V1
B4	rejet des faces par lots de 8, compaction par masque	V1	0 pixel différent ; gain sur V1
B5	faces de dos en espace objet	V1	gain contre B4 ; pixels différents comptés
B6	variante AVX-512, dispatch CPUID	V1 sur l'i9	même image ; gain sur l'i9
—	FMA : décision séparée	V1	nouvelle image de référence acceptée explicitement

Chaque pas suit la méthode : paire A/B dans un même lot, contrôles M3 et M19, V2 comme garde-fou (aucune régression quand les pixels dominent), et la scène active pour vérifier qu'elle ne bouge pas.

10. Les contre-exemples, rassemblés

> × Vectoriser l'intérieur d'un sommet (SIMD horizontal) (§ 3).

> × Garder l'AoS et compenser par des `gather` ou des shuffles (§ 4).

> × Convertir AoS → SoA à chaque frame (§ 4).

> × Une queue scalaire à côté de la boucle vectorielle : deux chemins, deux résultats possibles (§ 4).

> × Introduire FMA ou `rcp` en passant : c'est un changement d'image (§ 5.1).

> × Mesurer la vectorisation sur la Référence L, qui ne soumet plus que 1 388 faces : le gain serait invisible (A19 § 7).

> × Choisir AVX-512 par `#ifdef` (§ 8).

11. Aide-mémoire

Notion	Retenir
Ce qui se vectorise	la même opération sur 8 éléments **indépendants** et **contigus**
SIMD vertical	un registre = une composante de 8 sommets
Disposition	SoA, alignée sur 32 octets, bourrée à un multiple de 8, faite au chargement
Bit près	même ordre d'opérations que le scalaire ; division exacte ; FMA = décision séparée
Faces	tests vectoriels, dessin scalaire des survivantes (masque + `countr_zero`)
Faces de dos en objet	exact (sauf miroir, ortho) ; pas bit-exact avec l'aire en raster ; à mesurer contre B4

Notion	Retenir
Binaire	`/arch:AVX2` partout ; AVX-512 par CPUID à l'exécution
Mesure	campagne V : V1 (faces) pour le gain, V2 (pixels) pour la non-régression